SDK Examples and Workflows

Practical examples and real-world workflows for the Graph OLAP Python SDK.

Prerequisites

- 01-quickstart.manual.md - Getting started
- 02-client.manual.md - Client configuration
- 03-resources.manual.md - Resource management
- 04-queries.manual.md - Query execution
- 05-algorithms.manual.md - Algorithm reference

---

1. Common Workflows

Basic Workflow (Mapping -> Instance -> Query)

> **Note:** Snapshots are now managed internally. Instances are created directly from mappings.

The standard workflow creates a graph mapping, starts an instance (snapshot created internally), and executes queries.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

# Initialize client from environment variables
client = GraphOLAPClient.from_env()

# Create or get an existing mapping
mapping = client.mappings.get("customer-graph")

# Create an instance directly from mapping (snapshot managed internally)
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=mapping.id,
    name="Customer Analysis",
```

```
    wrapper_type=WrapperType.RYUGRAPH,
    timeout=600,  # Wait up to 10 minutes for export + startup
)

# Connect to the running instance
conn = client.instances.connect(instance.id)

# Execute queries
result = conn.query("MATCH (c:Customer) RETURN c.name, c.value ORDER BY c.value
DESC LIMIT 10")
df = result.to_pandas()
print(df)

# Always clean up when done
client.instances.terminate(instance.id)
client.close()
```

Quick Start Helper

For rapid exploration, use the client's `quick_start` method to create an instance from a mapping and connect in one call.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

client = GraphOLAPClient.from_env()

# Get a connection in one line (creates instance from mapping, snapshot managed
internally)
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# Start querying immediately
result = conn.query("MATCH (n) RETURN labels(n)[0] AS label, count(*) AS
count")
result.show()

# When done, terminate instance
client.instances.terminate(conn._instance_id)
client.close()
```

Using Existing Resources

If resources already exist, you can skip creation steps.

```python
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

client = GraphOLAPClient.from_env()

# Find an existing running instance by mapping
instances = client.instances.list(mapping_id=42, status="running")

if instances:
    # Reuse existing instance
    instance = instances[0]
else:
    # Create new instance directly from mapping (snapshot managed internally)
    # wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
    instance = client.instances.create_from_mapping_and_wait(
        mapping_id=42,
        name="Analysis Instance",
        wrapper_type=WrapperType.RYUGRAPH,
    )

conn = client.instances.connect(instance.id)
# ... perform analysis ...
```

Context Manager Pattern

Use context managers for automatic cleanup.

```python
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

with GraphOLAPClient.from_env() as client:
    # wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
    conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

    try:
        # Run your analysis
        result = conn.query("MATCH (n)-[r]->(m) RETURN n, r, m LIMIT 100")
        df = result.to_pandas()

    finally:
        # Always terminate to avoid resource leaks
        client.instances.terminate(conn._instance_id)
```

2. Real-World Use Cases

Customer Analysis

Identify influential customers using PageRank and community detection.

```python
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# ==============================================================================
# Step 1: Run PageRank to find influential customers
# ==============================================================================
exec_pr = conn.algo.pagerank(
    node_label="Customer",
    property_name="influence_score",
    damping=0.85,
    iterations=20
)
print(f"PageRank completed: {exec_pr.result['nodes_updated']} nodes scored")

# ==============================================================================
# Step 2: Detect communities using Louvain algorithm
# ==============================================================================
exec_louvain = conn.algo.louvain(
    node_label="Customer",
    property_name="community_id"
)
print(f"Found {exec_louvain.result['communities_found']} communities")

# ==============================================================================
# Step 3: Query top customers per community
# ==============================================================================
top_per_community = conn.query_df("""
    MATCH (c:Customer)
    WITH c.community_id AS community, c
    ORDER BY c.influence_score DESC
    WITH community, collect(c)[0..5] AS top_customers
    UNWIND top_customers AS customer
    RETURN
        community,
        customer.name AS name,
        customer.influence_score AS influence,
        customer.total_purchases AS purchases
    ORDER BY community, influence DESC
""")
```

```
print("\nTop Customers by Community:")
print(top_per_community)

# ==============================================================================
# Step 4: Visualize with PyVis
# ==============================================================================
# Get subgraph of top 100 customers
result = conn.query("""
    MATCH (c:Customer)
    WITH c ORDER BY c.influence_score DESC LIMIT 100
    MATCH (c)-[r:PURCHASED]->(p:Product)
    RETURN c, r, p
""")

# Convert to PyVis network
net = result.to_pyvis(
    notebook=True,
    height="600px",
    node_color_property="community_id",
    node_size_property="influence_score"
)
net.show("customer_network.html")

# Cleanup
client.instances.terminate(conn._instance_id)
client.close()
```

Fraud Detection

Identify suspicious patterns using centrality analysis and path finding.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=2, wrapper_type=WrapperType.RYUGRAPH)  #
Fraud detection graph

# ==============================================================================
# Step 1: Calculate betweenness centrality (money flow brokers)
# ==============================================================================
conn.networkx.run(
    algorithm="betweenness_centrality",
    node_label="Account",
    property_name="betweenness",
    params={"normalized": True}
)

# ==============================================================================
```

```python
# Step 2: Find accounts with unusually high centrality
# ==============================================================================
suspicious_accounts = conn.query_df("""
    MATCH (a:Account)
    WHERE a.betweenness > 0.1
    RETURN
        a.account_id AS account,
        a.holder_name AS holder,
        a.betweenness AS centrality,
        a.account_age_days AS age_days,
        size((a)-[:TRANSFER]->()) AS outgoing_transfers,
        size((a)<-[:TRANSFER]-()) AS incoming_transfers
    ORDER BY a.betweenness DESC
    LIMIT 20
""")

print("High-Centrality Accounts (Potential Money Brokers):")
print(suspicious_accounts)

# ==============================================================================
# Step 3: Find circular transaction patterns (potential layering)
# ==============================================================================
circular_patterns = conn.query_df("""
    MATCH path = (a:Account)-[:TRANSFER*3..6]->(a)
    WHERE all(t IN relationships(path) WHERE t.amount > 1000)
    WITH a, path,
         reduce(total = 0, t IN relationships(path) | total + t.amount) AS
total_amount
    RETURN DISTINCT
        a.account_id AS origin_account,
        length(path) AS cycle_length,
        total_amount,
        [n IN nodes(path) | n.account_id] AS accounts_in_cycle
    ORDER BY total_amount DESC
    LIMIT 10
""")

print("\nCircular Transaction Patterns:")
print(circular_patterns)

# ==============================================================================
# Step 4: Path analysis between known fraudulent entities
# ==============================================================================
# Find shortest path between two suspicious accounts
path_result = conn.algo.shortest_path(
    source_id="ACC_SUSPICIOUS_001",
    target_id="ACC_SUSPICIOUS_002",
    weight_property="amount"
)

if path_result:
    print(f"\nPath between suspicious accounts: {path_result['path']}")
    print(f"Total value transferred: ${path_result['total_weight']:,.2f}")
```

```
# ==============================================================================
# Step 5: Anomaly detection - accounts with unusual patterns
# ==============================================================================
anomalies = conn.query_df("""
    MATCH (a:Account)
    WITH a,
         size((a)-[:TRANSFER]->()) AS out_count,
         size((a)<-[:TRANSFER]-()) AS in_count
    WHERE out_count > 0 AND in_count > 0
    WITH a, out_count, in_count,
         toFloat(out_count) / in_count AS ratio
    WHERE ratio > 10 OR ratio < 0.1  // Highly asymmetric
    RETURN
        a.account_id AS account,
        a.holder_name AS holder,
        out_count,
        in_count,
        ratio,
        a.betweenness AS centrality
    ORDER BY a.betweenness DESC
    LIMIT 20
""")

print("\nAnomalous Transfer Patterns:")
print(anomalies)

# Cleanup
client.instances.terminate(conn._instance_id)
client.close()
```

Network Analysis

Analyze influence propagation, link prediction, and clustering.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
import polars as pl

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=3, wrapper_type=WrapperType.RYUGRAPH)  #
Social network graph

# ==============================================================================
# Step 1: Calculate multiple centrality metrics
# ==============================================================================
# Degree centrality (popularity)
conn.networkx.run(
    algorithm="degree_centrality",
    node_label="User",
```

```
    property_name="degree_cent"
)

# Closeness centrality (reach)
conn.networkx.run(
    algorithm="closeness_centrality",
    node_label="User",
    property_name="closeness_cent"
)

# Eigenvector centrality (influence)
conn.networkx.run(
    algorithm="eigenvector_centrality",
    node_label="User",
    property_name="eigen_cent",
    params={"max_iter": 100}
)

# ==============================================================================
# Step 2: Identify key influencers
# ==============================================================================
influencers = conn.query_df("""
    MATCH (u:User)
    RETURN
        u.username AS user,
        u.follower_count AS followers,
        u.degree_cent AS degree,
        u.closeness_cent AS closeness,
        u.eigen_cent AS eigenvector,
        (u.degree_cent + u.closeness_cent + u.eigen_cent) / 3 AS combined_score
    ORDER BY combined_score DESC
    LIMIT 20
""")

print("Top Influencers (Multi-metric Analysis):")
print(influencers)

# ==============================================================================
# Step 3: Link prediction - who should connect?
# ==============================================================================
link_predictions = conn.networkx.run(
    algorithm="jaccard_coefficient",
    node_label="User",
    edge_types=["FOLLOWS"]
)

# Top predicted connections
print("\nPredicted Connections (Jaccard Coefficient):")
for pred in link_predictions.result["predictions"][:10]:
    print(f"  {pred['source']} <-> {pred['target']}: {pred['score']:.3f}")

# ==============================================================================
# Step 4: Clustering coefficient (local connectivity)
# ==============================================================================
```

```python
conn.networkx.run(
    algorithm="clustering",
    node_label="User",
    property_name="clustering_coeff"
)

# Find users who bridge different groups (low clustering, high centrality)
bridge_users = conn.query_df("""
    MATCH (u:User)
    WHERE u.clustering_coeff < 0.2 AND u.eigen_cent > 0.1
    RETURN
        u.username AS user,
        u.clustering_coeff AS clustering,
        u.eigen_cent AS influence,
        size((u)-[:FOLLOWS]->()) AS following,
        size((u)<-[:FOLLOWS]-()) AS followers
    ORDER BY u.eigen_cent DESC
    LIMIT 15
""")

print("\nBridge Users (Low Clustering, High Influence):")
print(bridge_users)

# ==============================================================================
# Step 5: Community structure analysis
# ==============================================================================
conn.networkx.run(
    algorithm="louvain_communities",
    property_name="community",
    params={"resolution": 1.0}
)

community_stats = conn.query_df("""
    MATCH (u:User)
    WITH u.community AS community, collect(u) AS members
    RETURN
        community,
        size(members) AS member_count,
        avg([m IN members | m.eigen_cent][0]) AS avg_influence,
        max([m IN members | m.follower_count][0]) AS max_followers
    ORDER BY member_count DESC
    LIMIT 10
""")

print("\nCommunity Statistics:")
print(community_stats)

# Cleanup
client.instances.terminate(conn._instance_id)
client.close()
```

3. Integration Patterns

Pandas/Polars Workflows

Seamless integration with DataFrame libraries for analysis pipelines.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
import pandas as pd
import polars as pl

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# ============================================================================
# Pandas Integration
# ============================================================================

# Get data as Pandas DataFrame
df_pandas = conn.query("MATCH (c:Customer) RETURN c.* LIMIT 1000").to_pandas()

# Apply Pandas transformations
df_pandas["revenue_tier"] = pd.cut(
    df_pandas["total_revenue"],
    bins=[0, 1000, 10000, 100000, float("inf")],
    labels=["Bronze", "Silver", "Gold", "Platinum"]
)

# Group and aggregate
summary = df_pandas.groupby("revenue_tier").agg({
    "customer_id": "count",
    "total_revenue": ["sum", "mean"],
    "order_count": "sum"
}).round(2)

print("Customer Revenue Tiers (Pandas):")
print(summary)

# ============================================================================
# Polars Integration (Faster for large datasets)
# ============================================================================

# Get data as Polars DataFrame
df_polars = conn.query("MATCH (c:Customer) RETURN c.* LIMIT
100000").to_polars()

# Apply Polars transformations (lazy evaluation)
result = (
    df_polars
```

```
    .lazy()
    .with_columns([
        pl.when(pl.col("total_revenue") > 100000).then(pl.lit("Platinum"))
        .when(pl.col("total_revenue") > 10000).then(pl.lit("Gold"))
        .when(pl.col("total_revenue") > 1000).then(pl.lit("Silver"))
        .otherwise(pl.lit("Bronze"))
        .alias("tier")
    ])
    .group_by("tier")
    .agg([
        pl.count().alias("customer_count"),
        pl.sum("total_revenue").alias("total_revenue"),
        pl.mean("order_count").alias("avg_orders")
    ])
    .sort("total_revenue", descending=True)
    .collect()
)

print("\nCustomer Revenue Tiers (Polars):")
print(result)

# ==============================================================================
# Combining Graph Queries with DataFrame Operations
# ==============================================================================

# Get graph metrics
conn.algo.pagerank("Customer", "pr_score")
conn.networkx.run("clustering", node_label="Customer",
property_name="clustering")

# Query combined metrics
metrics_df = conn.query_df("""
    MATCH (c:Customer)
    RETURN
        c.customer_id AS id,
        c.name AS name,
        c.total_revenue AS revenue,
        c.pr_score AS pagerank,
        c.clustering AS clustering_coeff
""", use_polars=True)

# Polars analysis
top_customers = (
    metrics_df
    .with_columns([
        (pl.col("pagerank") * pl.col("revenue")).alias("weighted_influence")
    ])
    .sort("weighted_influence", descending=True)
    .head(20)
)

print("\nTop Customers by Weighted Influence:")
print(top_customers)
```

```
# Cleanup
client.instances.terminate(conn._instance_id)
client.close()
```

NetworkX Visualization

Export graph data to NetworkX for advanced visualization and analysis.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
import networkx as nx
import matplotlib.pyplot as plt

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# ============================================================================
# Export Subgraph to NetworkX
# ============================================================================

# Query a subgraph (nodes and edges)
result = conn.query("""
    MATCH (c:Customer)-[p:PURCHASED]->(pr:Product)
    WHERE c.region = 'EMEA'
    RETURN c, p, pr
    LIMIT 500
""")

# Convert to NetworkX graph
G = result.to_networkx(directed=True)

print(f"Exported graph: {G.number_of_nodes()} nodes, {G.number_of_edges()}
edges")

# ============================================================================
# NetworkX Analysis (client-side)
# ============================================================================

# Calculate metrics using NetworkX
nx_pagerank = nx.pagerank(G)
nx_betweenness = nx.betweenness_centrality(G)
nx_communities = list(nx.community.louvain_communities(G.to_undirected()))

print(f"\nLocal NetworkX Analysis:")
print(f"  PageRank computed for {len(nx_pagerank)} nodes")
print(f"  Found {len(nx_communities)} communities")

# ============================================================================
# Matplotlib Visualization
```

```python
# ==============================================================================

# Create layout
pos = nx.spring_layout(G, k=0.5, iterations=50)

# Color nodes by type
node_colors = []
for node in G.nodes():
    label = G.nodes[node].get("_label", "Unknown")
    if label == "Customer":
        node_colors.append("#1f77b4")
    elif label == "Product":
        node_colors.append("#ff7f0e")
    else:
        node_colors.append("#7f7f7f")

# Size nodes by PageRank
node_sizes = [nx_pagerank.get(n, 0.01) * 3000 for n in G.nodes()]

# Draw graph
plt.figure(figsize=(14, 10))
nx.draw(
    G, pos,
    node_color=node_colors,
    node_size=node_sizes,
    edge_color="#cccccc",
    alpha=0.7,
    with_labels=False,
    arrows=True,
    arrowsize=8
)
plt.title("Customer-Product Network (Node size = PageRank)")
plt.tight_layout()
plt.savefig("network_visualization.svg", dpi=150)
plt.show()

# ==============================================================================
# PyVis Interactive Visualization
# ==============================================================================

# Alternative: use built-in PyVis conversion
net = result.to_pyvis(
    notebook=True,
    height="700px",
    width="100%"
)

# Customize appearance
net.barnes_hut(gravity=-3000, central_gravity=0.3, spring_length=200)
net.show_buttons(filter_=["physics"])
net.show("interactive_network.html")

# Cleanup
```

```
client.instances.terminate(conn._instance_id)
client.close()
```

Exporting to CSV/Parquet

Export query results and analysis data for downstream processing.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
from pathlib import Path

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# Create output directory
output_dir = Path("./exports")
output_dir.mkdir(exist_ok=True)

# ==============================================================================
# Export Query Results to CSV
# ==============================================================================

# Run algorithms first
conn.algo.pagerank("Customer", "pr_score")
conn.algo.louvain("Customer", "community_id")

# Query and export to CSV
result = conn.query("""
    MATCH (c:Customer)
    RETURN
        c.customer_id AS id,
        c.name AS name,
        c.email AS email,
        c.region AS region,
        c.total_revenue AS revenue,
        c.pr_score AS pagerank,
        c.community_id AS community
    ORDER BY c.pr_score DESC
""")

# Export to CSV
result.to_csv(str(output_dir / "customers_with_metrics.csv"))
print(f"Exported {result.row_count} customers to CSV")

# ==============================================================================
# Export to Parquet (better for large datasets)
# ==============================================================================

# Export all transactions
```

```
transactions = conn.query("""
    MATCH (c:Customer)-[t:PURCHASED]->(p:Product)
    RETURN
        c.customer_id AS customer_id,
        p.product_id AS product_id,
        t.amount AS amount,
        t.quantity AS quantity,
        t.transaction_date AS date
""")

# Parquet with compression
transactions.to_parquet(
    str(output_dir / "transactions.parquet"),
    compression="snappy"
)
print(f"Exported {transactions.row_count} transactions to Parquet")

# ==============================================================================
# Export Graph Structure (Nodes and Edges)
# ==============================================================================

# Export nodes
nodes_df = conn.query_df("""
    MATCH (n)
    RETURN
        id(n) AS node_id,
        labels(n)[0] AS label,
        properties(n) AS properties
""")
nodes_df.write_parquet(str(output_dir / "nodes.parquet"))

# Export edges
edges_df = conn.query_df("""
    MATCH ()-[r]->()
    RETURN
        id(startNode(r)) AS source_id,
        id(endNode(r)) AS target_id,
        type(r) AS edge_type,
        properties(r) AS properties
""")
edges_df.write_parquet(str(output_dir / "edges.parquet"))

print(f"Exported graph structure: {len(nodes_df)} nodes, {len(edges_df)}
edges")

# ==============================================================================
# Export to JSON (for web applications)
# ==============================================================================

# Export community summary as JSON
community_summary = conn.query("""
    MATCH (c:Customer)
    WITH c.community_id AS community, collect(c.name) AS members, count(*) AS
size
```

```
    RETURN community, size, members[0..5] AS sample_members
    ORDER BY size DESC
    LIMIT 20
""")

community_summary.to_json(str(output_dir / "communities.json"), indent=2)
print("Exported community summary to JSON")

# ============================================================================
# Batch Export with Progress
# ============================================================================

def export_with_progress(conn, query, output_path, batch_size=10000):
    """Export large result sets in batches with progress tracking."""

    # First, get total count
    count_query = f"MATCH (n) RETURN count(n) AS total"
    total = conn.query_scalar(count_query)

    print(f"Exporting {total} records...")

    offset = 0
    batch_num = 0

    while offset < total:
        batch_query = f"{query} SKIP {offset} LIMIT {batch_size}"
        result = conn.query(batch_query)

        if batch_num == 0:
            # First batch - write with header
            result.to_csv(output_path)
        else:
            # Append without header
            df = result.to_polars()
            with open(output_path, "a") as f:
                df.write_csv(f, include_header=False)

        offset += batch_size
        batch_num += 1
        progress = min(100, (offset / total) * 100)
        print(f"  Progress: {progress:.1f}%")

    print(f"Export complete: {output_path}")

# Example usage
export_with_progress(
    conn,
    "MATCH (c:Customer) RETURN c.*",
    str(output_dir / "all_customers.csv"),
    batch_size=5000
)

# Cleanup
```

```
client.instances.terminate(conn._instance_id)
client.close()
```

---

4. Advanced Patterns

Temporal Analysis

Analyze time-series patterns in graph data.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
from datetime import datetime, timedelta

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# ==============================================================================
# Time-windowed Analysis
# ==============================================================================

# Get transaction patterns by month
monthly_patterns = conn.query_df("""
    MATCH (c:Customer)-[t:PURCHASED]->(p:Product)
    WITH
        date(t.transaction_date).year AS year,
        date(t.transaction_date).month AS month,
        c, t, p
    RETURN
        year, month,
        count(DISTINCT c) AS unique_customers,
        count(t) AS transaction_count,
        sum(t.amount) AS total_revenue,
        avg(t.amount) AS avg_transaction
    ORDER BY year, month
""")

print("Monthly Transaction Patterns:")
print(monthly_patterns)

# ==============================================================================
# Cohort Analysis
# ==============================================================================

# Analyze customers by signup cohort
cohort_analysis = conn.query_df("""
```

```
    MATCH (c:Customer)
    WITH
        date(c.signup_date).year AS cohort_year,
        date(c.signup_date).month AS cohort_month,
        c
    OPTIONAL MATCH (c)-[t:PURCHASED]->()
    WITH cohort_year, cohort_month, c, count(t) AS purchases
    RETURN
        cohort_year, cohort_month,
        count(c) AS cohort_size,
        avg(purchases) AS avg_purchases,
        sum(CASE WHEN purchases > 0 THEN 1 ELSE 0 END) AS active_customers
    ORDER BY cohort_year, cohort_month
""")

print("\nCohort Analysis:")
print(cohort_analysis)

# Cleanup
client.instances.terminate(conn._instance_id)
client.close()
```

Multi-hop Path Analysis

Analyze relationship chains and influence paths.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# ==============================================================================
# Find Influence Chains
# ==============================================================================

# Find paths from top influencer to others
influence_paths = conn.query_df("""
    MATCH (top:Customer)
    WHERE top.pr_score > 0.1
    WITH top ORDER BY top.pr_score DESC LIMIT 5
    MATCH path = (top)-[:REFERRED*1..3]->(reached:Customer)
    RETURN
        top.name AS influencer,
        reached.name AS reached_customer,
        length(path) AS path_length,
        [n IN nodes(path) | n.name] AS path_names
    ORDER BY top.pr_score DESC, path_length
    LIMIT 50
```

```
""")

print("Influence Chains from Top Customers:")
print(influence_paths)

# ==============================================================================
# Supply Chain Analysis
# ==============================================================================

# Find all paths from suppliers to end customers
supply_paths = conn.query_df("""
    MATCH path = (s:Supplier)-[:SUPPLIES*]->(d:Distributor)-[:SELLS_TO*]->
(c:Customer)
    WHERE length(path) <= 5
    WITH s, c, path,
         [r IN relationships(path) | r.lead_time_days] AS lead_times
    RETURN
        s.name AS supplier,
        c.name AS customer,
        length(path) AS hops,
        reduce(total = 0, lt IN lead_times | total + lt) AS total_lead_time
    ORDER BY total_lead_time
    LIMIT 20
""")

print("\nSupply Chain Paths:")
print(supply_paths)

# Cleanup
client.instances.terminate(conn._instance_id)
client.close()
```

Hybrid Analysis (Graph + SQL)

Combine graph analysis with external SQL data.

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
import pandas as pd

client = GraphOLAPClient.from_env()
# wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-memory)
conn = client.quick_start(mapping_id=1, wrapper_type=WrapperType.RYUGRAPH)

# ==============================================================================
# Get Graph Metrics
# ==============================================================================

# Calculate graph-based scores
conn.algo.pagerank("Customer", "graph_influence")
```

```
conn.algo.louvain("Customer", "community")

# Export customer metrics from graph
graph_metrics = conn.query_df("""
    MATCH (c:Customer)
    RETURN
        c.customer_id AS customer_id,
        c.graph_influence AS influence_score,
        c.community AS community_id,
        size((c)-[:PURCHASED]->()) AS product_diversity
""").to_pandas()

# ============================================================================
# Load External Data (e.g., from data warehouse)
# ============================================================================

# Simulate loading CRM data from external source
crm_data = pd.DataFrame({
    "customer_id": graph_metrics["customer_id"].tolist(),
    "ltv": [1000 * (i + 1) for i in range(len(graph_metrics))],
    "churn_risk": [0.1 + 0.05 * (i % 10) for i in range(len(graph_metrics))],
    "segment": ["Enterprise" if i % 5 == 0 else "SMB" for i in
range(len(graph_metrics))]
})

# ============================================================================
# Combine Graph and SQL Metrics
# ============================================================================

# Merge datasets
combined = graph_metrics.merge(crm_data, on="customer_id")

# Create composite scores
combined["retention_priority"] = (
    combined["influence_score"] * 0.3 +
    (combined["ltv"] / combined["ltv"].max()) * 0.4 +
    combined["churn_risk"] * 0.3
)

# Segment analysis
segment_analysis = combined.groupby("segment").agg({
    "customer_id": "count",
    "influence_score": "mean",
    "ltv": "sum",
    "churn_risk": "mean",
    "retention_priority": "mean"
}).round(3)

print("Hybrid Analysis - Segment Summary:")
print(segment_analysis)

# Top retention targets
top_targets = combined.nlargest(10, "retention_priority")[
    ["customer_id", "segment", "influence_score", "ltv", "churn_risk",
```

```
"retention_priority"]
]

print("\nTop Retention Targets:")
print(top_targets)

# Cleanup
client.instances.terminate(conn._instance_id)
client.close()
```

---

5. Error Handling Patterns

Robust Workflow with Retries

```
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
from graph_olap.exceptions import (
    SnapshotFailedError,
    InstanceFailedError,
    QueryTimeoutError,
    ResourceLockedError
)
import time

def robust_analysis(client, mapping_id, wrapper_type, max_retries=3):
    """Execute analysis with comprehensive error handling.

    Args:
        client: GraphOLAPClient instance
        mapping_id: Mapping ID to use
        wrapper_type: WrapperType.RYUGRAPH (disk-backed) or
WrapperType.FALKORDB (in-memory)
        max_retries: Number of retries for failed operations
    """

    instance = None

    try:
        # Create instance from mapping with retry (snapshot managed internally)
        # wrapper_type is REQUIRED: RYUGRAPH (disk-backed) or FALKORDB (in-
memory)
        for attempt in range(max_retries):
            try:
                instance = client.instances.create_from_mapping_and_wait(
                    mapping_id=mapping_id,
                    name=f"Robust Analysis {time.strftime('%Y%m%d_%H%M%S')}",
```

```
                    wrapper_type=wrapper_type,
                    timeout=600
                )
                break
            except InstanceFailedError as e:
                if attempt < max_retries - 1:
                    print(f"Instance creation failed, retrying... ({e})")
                    time.sleep(10)
                else:
                    raise

        conn = client.instances.connect(instance.id)

        # Run algorithms with lock handling
        try:
            conn.algo.pagerank("Customer", "pr_score")
        except ResourceLockedError as e:
            print(f"Instance locked by {e.holder_name}, waiting...")
            time.sleep(30)
            conn.algo.pagerank("Customer", "pr_score")

        # Execute queries with timeout handling
        try:
            result = conn.query(
                "MATCH (c:Customer) RETURN c.* ORDER BY c.pr_score DESC LIMIT
100",
                timeout_ms=30000
            )
            return result.to_pandas()
        except QueryTimeoutError:
            print("Query timed out, trying simpler version...")
            result = conn.query(
                "MATCH (c:Customer) RETURN c.name, c.pr_score ORDER BY
c.pr_score DESC LIMIT 100",
                timeout_ms=60000
            )
            return result.to_pandas()

    finally:
        # Always clean up
        if instance:
            try:
                client.instances.terminate(instance.id)
            except Exception as e:
                print(f"Warning: cleanup failed: {e}")


# Usage
client = GraphOLAPClient.from_env()
try:
    df = robust_analysis(client, mapping_id=1,
wrapper_type=WrapperType.RYUGRAPH)
    print(df)
```

```
finally:
    client.close()
```

---

6. Performance Tips

Efficient Query Patterns

```
# BAD: Returns entire nodes when you only need a few properties
result = conn.query("MATCH (c:Customer) RETURN c")

# GOOD: Project only needed properties
result = conn.query("MATCH (c:Customer) RETURN c.id, c.name, c.value")

# BAD: Unbounded query
result = conn.query("MATCH (c:Customer)-[r]->(p:Product) RETURN c, r, p")

# GOOD: Always use LIMIT for exploration
result = conn.query("MATCH (c:Customer)-[r]->(p:Product) RETURN c, r, p LIMIT
1000")

# BAD: Multiple sequential queries
for customer_id in customer_ids:
    result = conn.query(f"MATCH (c:Customer {{id: '{customer_id}'}}) RETURN c")

# GOOD: Batch with UNWIND
result = conn.query("""
    UNWIND $ids AS id
    MATCH (c:Customer {id: id})
    RETURN c
""", parameters={"ids": customer_ids})

# GOOD: Use parameters for dynamic values (prevents injection, enables caching)
result = conn.query(
    "MATCH (c:Customer) WHERE c.region = $region RETURN c LIMIT $limit",
    parameters={"region": "EMEA", "limit": 100}
)
```

Algorithm Execution Tips

```
# Run algorithms on subgraphs when possible
conn.algo.pagerank(
    node_label="Customer",  # Filter to specific label
```

```python
    property_name="pr_score",
)

# Use sampling for expensive algorithms on large graphs
conn.networkx.run(
    algorithm="betweenness_centrality",
    node_label="Customer",
    property_name="bc",
    params={"k": 500}  # Sample 500 nodes instead of all
)

# Monitor algorithm progress
exec = conn.algo.pagerank("Customer", "pr_score", wait=False)
while True:
    status = conn.algo.status(exec.execution_id)
    print(f"Status: {status.status}, Progress: {status.progress}%")
    if status.status in ("completed", "failed"):
        break
    time.sleep(5)
```